SCBE (Semi Computer Based Evaluation)

Project Report — Prototype Stage, for UCS503P
Submitted to: Dr. Raghav B. Venkataramaiyer
Thapar Institute of Engineering and Technology

Team Infinix

Gurneet Singh Ahuja, CSED* Alla Harshith, CSED† Arpit Goyal, CSED‡

September 8, 2026

Contents

1 Introduction **2**
1.1 Objectives: status at the prototype stage 2

2 System Architecture **3**
2.1 Core workflow as implemented 3
2.2 Technical stack 4

3 Module-by-Module Implementation **4**
3.1 Video/image-to-PDF conversion 4
3.2 Backend orchestration and data model 5
3.3 Annotation and complaint capture 5
3.4 Faculty and student dashboards 5

4 Testing and Validation **5**
4.1 Result: what currently works vs. what does not 6

5 Challenges and Mitigations **7**

6 Limitations **7**

7 Conclusion and Future Work **7**

*Roll No: 1024170014, Email: gahuja_be24@thapar.edu
†Roll No: 1024170007, Email: aharshith_be24@thapar.edu
‡Roll No: 1024170021, Email: agoyal6_be24@thapar.edu

1 Introduction

SCBE (Semi Computer Based Evaluation) is a platform for digitizing the post-examination answer-sheet review process. In the workflow the platform targets, faculty capture a student's physical answer booklet as a short video or a set of photographs; the platform assembles these captures into a single PDF, lets faculty attach the sheet to the correct student and examination, and publishes it so the student can view and annotate their own evaluated answer sheet remotely. If a student disagrees with part of the evaluation, they can raise a complaint directly on the annotated page, and the faculty member can review and resolve it from their own dashboard. The proposal framed this as a way to cut down on the travel, delay, and lack of a paper trail that come with the existing practice of physically re-checking answer sheets on campus.

Since the proposal, the team has built a working three-tier prototype – a Python/OpenCV service that turns raw video or images into a paginated PDF, a Node.js/Express backend that owns authentication, data persistence and orchestration, and a React frontend with role-specific dashboards for students, faculty, and an examination-cell admin. This report describes what has actually been implemented, how the pieces fit together, and what has changed relative to the original plan.

1.1 Objectives: status at the prototype stage

Objective (from proposal)	**Status**	**Notes**
Faculty and student authentication with access controls	Met	Role-based login/registration (student, faculty, admin) with salted, hashed passwords; routes are scoped per role and per record owner.
Faculty workflow for uploading and managing evaluated answer sheets	Met	Video-to-PDF and images-to-PDF conversion, plus page-level add/delete/replace editing, are implemented end to end.
Student interface for accessing and viewing evaluated sheets	Met	In-browser PDF viewer with per-question mark totals and evaluation breakdown.
Student workflow for raising review requests	Met	Students annotate directly on the PDF and submit a complaint tied to a specific question and the annotated page.
Faculty workflow for reviewing and responding to requests	Met	Faculty can inspect the annotated PDF, change complaint status (Pending / Under Review / Resolved / Rejected), and attach a comment.
Recording of requests, responses, and resolutions	Met	Persisted in a relational schema (complaints, discrepancies, notifications) with an audit trail of status changes.
CI/CD with automated testing to a hosted staging environment	Not started	Deferred; the prototype currently runs from local processes only. See Section 6.

Objective (from proposal)	Status	Notes
Automatic association of answer sheets with the correct student (OCR)	Partially met	The database and API already carry `detectedRollNumber` / `detectedName` fields, but the OCR/handwriting-recognition step that should populate them has not been built yet; faculty currently assign each sheet to a student manually.
Faculty verification/correction of automatic associations	Deferred	Depends on the OCR step above; the manual assignment screen that will later serve as the "correction" step already exists.

The most significant deviation from the proposal is on automatic answer-sheet identification. The proposal treated OCR-based name/roll-number extraction as part of the initial deliverables; in practice the team prioritized getting the full manual pipeline – capture, convert, assign, publish, review, complain, resolve – working reliably first, and pushed OCR to the next iteration. This is discussed further in Section 5.

2 System Architecture

The prototype is organised as three independently runnable services that communicate over HTTP, matching the decoupled, stateless design the proposal argued for on scalability grounds.

```
React frontend (Vite) ⟶ Node.js/Express backend (port 4000) ⟶ Flask/OpenCV
                                CV service (port 5000)
```

- **Frontend (React + Vite):** role-specific pages for students, faculty, and admins, an in-browser PDF viewer/annotator (`@embedpdf/react-pdf-viewer`), and a thin `api.js` client that talks to the backend.
- **Backend (Node.js + Express):** authentication, the `users`/`courses`/`exams`/`sheets`/`complaints`/`discrepancies`/`notifications` data model, file storage for uploaded and generated documents, and a thin proxy layer that forwards capture jobs to the CV service.
- **CV service (Python + Flask + OpenCV):** stateless image/video-processing routes that turn a video or a set of images into a PDF, and that perform page-level edits (delete/add/replace) on an existing PDF.
- **Storage:** a single SQLite database (`scbe.sqlite`, in WAL mode) for structured data, and a directory tree on disk for uploaded videos/images and generated original/annotated PDFs.

2.1 Core workflow as implemented

1. Faculty record a short video (or take photos) of the evaluated answer booklet and upload it from the *Upload* screen.
2. The backend streams the file to the CV service, which returns a generated PDF; the backend stores the PDF and creates an unassigned `sheet` row.

3. From the *Assign* screen, faculty attach the sheet to a student and an exam (roll number/name can currently be entered manually; see Section 6 on OCR).

4. Faculty publish the sheet, which triggers a notification to the student and makes the sheet visible on their dashboard.

5. The student opens the sheet in the PDF viewer, sees the per-question marks and total, and can annotate a page and raise a complaint about a specific question.

6. The complaint (with the flattened annotated PDF and the raw annotation JSON) lands on the faculty's complaint queue; the faculty member reviews it in the same viewer, changes its status, and adds a resolution comment, which the student can then see.

2.2 Technical stack

Component	**Technology**
Frontend	React 18, React Router, Vite
PDF viewing/annotation	`@embedpdf/react-pdf-viewer` (PDFium/WASM)
Backend	Node.js, Express, Multer (file uploads)
Database	SQLite (`node:sqlite`, WAL mode)
Password storage	`scrypt` key derivation with per-user salt
CV / document assembly	Python, Flask, Flask-CORS
Frame processing	OpenCV (`opencv-python-headless`), NumPy
Image/PDF assembly	Pillow, PyPDF2

Table 2: Technical stack used in the prototype

3 Module-by-Module Implementation

3.1 Video/image-to-PDF conversion

This is the module that most directly replaces the proposal's "dedicated assistant scans the sheets" step. Rather than requiring faculty to pick out individual page photographs by hand, the CV service accepts a single video of the whole booklet being flipped through and extracts one clean frame per page.

The extraction logic samples the incoming video at a fixed rate (default 20 fps) and tracks a simple two-state machine per sampled frame:

- **STABLE:** the camera/page is holding still. Every sampled frame is kept as a candidate for the current page, scored by a Laplacian-variance sharpness measure (a common focus-quality heuristic: a sharper image has more high-frequency edge content, hence higher Laplacian variance).
- **FLIPPING:** consecutive frame-to-frame differences (mean absolute pixel difference on grayscale frames) exceed a threshold for enough consecutive samples, which is taken as evidence that a page turn is in progress.

When the frame difference stays low for long enough after a flip, the state returns to STABLE and a new page begins accumulating candidates. When a flip is detected, the sharpest candidate frame collected during the previous STABLE run is committed as a finished page. All committed frames are finally stitched, in order, into a single multi-page PDF. Sample rate, the difference

threshold, and the number of consecutive frames required to confirm a flip or a stabilization are all exposed as tunable parameters on the API rather than hard-coded, which was useful during testing under different lighting and camera-shake conditions.

The same service also accepts a batch of individual page images directly (for faculty who prefer photographing pages one at a time) and stitches them into a PDF in upload order, and it can delete, insert, or replace a single page of an already-generated PDF – used when a page was mis-captured and needs to be redone without regenerating the whole document.

3.2 Backend orchestration and data model

The Node backend does not perform any image processing itself; it is responsible for authentication, persistence, and gluing the frontend to the CV service. Uploaded video/image files are received with Multer, forwarded to the CV service as `multipart/form-data`, and the resulting PDF is copied into a durable `generated/pdfs/originals` directory referenced by a new `sheets` row.

The schema (SQLite) currently has seven tables: `users`, `courses`, `exams`, `sheets`, `discrepancies`, `notifications`, `complaints`, and a small `meta` table used for lightweight schema-version bookkeeping. The `sheets` table already carries `detectedRollNumber` and `detectedName` columns, reflecting the planned OCR integration, even though nothing writes to them automatically yet – they are filled in by faculty through the manual assignment endpoint. Passwords are never stored in plaintext: each user gets a random salt, and the derived `scrypt` hash is compared using a constant-time comparison to reduce timing side-channels.

3.3 Annotation and complaint capture

Students view their sheet using `@embedpdf/react-pdf-viewer`, which runs PDF rendering through a PDFium WebAssembly build in the browser rather than round-tripping page images through the server. When a student opens the complaint form on a page, the viewer's annotation plugin exports the annotation set for that document (through its `exportAnnotations` capability); the frontend attaches this JSON alongside the already-flattened, annotated PDF and the complaint title/description, and posts it as a single multipart request. This gives the faculty member two views of the same concern when they open the complaint later: the flattened PDF (a durable record that cannot be re-edited after submission) and the structured annotation data (useful if the interface ever needs to re-render annotations interactively instead of only showing the flattened image).

3.4 Faculty and student dashboards

Faculty have four working screens: a dashboard summarising counts and recent resolved complaints, a sheets list (assigned/unassigned, with per-sheet publish and reassignment actions), the capture/upload screen described above, and a complaint inbox with status transitions (*Pending* $\rightarrow$ *Under Review* $\rightarrow$ *Resolved*/*Rejected*). Students have a matching set: dashboard, sheet list, the annotated PDF viewer with the complaint form, a list of their own submitted complaints with status, and notifications. The admin role currently exposes read-only aggregate views (student/faculty counts, examinations, and a global list of answer sheets) for the examination cell to get a bird's-eye view of platform activity; write operations for admins (e.g. course/exam management from the UI) are not yet implemented.

4 Testing and Validation

Formal automated CI/CD and a load-tested staging deployment were listed in the proposal as initial deliverables but have not been reached at this stage; testing so far has been manual

and workflow-oriented rather than based on an automated test suite. The team exercised the following scenarios directly against the running services:

- **End-to-end happy path:** register a faculty and a student account, upload a sample video of a mock answer booklet, confirm a PDF is generated, assign it to the student, publish it, and confirm it appears on the student's dashboard.
- **Page-extraction quality:** varied the sample video's flip speed and lighting to check that the difference-threshold/sharpness heuristic still picks one legible frame per page; very fast flips or low light occasionally required raising `sample_fps` or lowering `difference_threshold` for reliable page separation.
- **PDF page editing:** deleted, inserted, and replaced a page on an already-generated PDF and confirmed the page count and page order in the output.
- **Complaint round-trip:** annotated a sheet as a student, submitted a complaint, confirmed the faculty inbox received it with the flattened PDF intact, changed its status with a comment, and confirmed the status/comment appeared back on the student side.
- **Access control:** confirmed a student cannot open another student's sheet or complain about a sheet that is not theirs, and that role-scoped routes reject requests for a user id/role that do not match the record (e.g. a student id used against a faculty-only route).

No formal performance benchmarking (throughput under concurrent uploads, response time under load) has been carried out yet; the proposal's "prototype workload testing" and "document delivery performance" items under Practical Scalability remain future work, as does the hosted deployment needed to test under realistic network conditions.

4.1 Result: what currently works vs. what does not

Capability	Result	Notes
Video → PDF conversion	Pass	Reliable for moderate flip speed and adequate lighting.
Images → PDF conversion	Pass	Direct, order-preserving stitching.
PDF page delete/add/replace	Pass	Verified against multi-page sample PDFs.
Manual sheet assignment & publish	Pass	Notification is created and appears for the student.
Student annotation & complaint submission	Pass	Flattened PDF and annotation JSON both persisted.
Faculty complaint review/resolution	Pass	Status and resolution comment both round-trip to the student view.
Per-student/per-role access control	Pass (spot-checked)	No automated authorization test suite yet.
Automatic student identification (OCR)	Not implemented	Field exists in schema; extraction pipeline not built.
CI/CD and hosted deployment	Not implemented	Prototype currently runs from local processes only.

5 Challenges and Mitigations

- **Separating a genuine page turn from camera shake.** An early version of the video-processing heuristic mistook ordinary camera wobble for a page flip, which fragmented single pages into several spurious "pages". Requiring several consecutive frames to cross the difference threshold before committing to a flip (rather than reacting to a single frame) removed most of the false positives without needing a heavier motion-tracking model.
- **Choosing a representative frame per page.** Even within a stable interval, individual frames vary in focus. Scoring every candidate frame with a Laplacian-sharpness measure and keeping the sharpest one per page gave a consistent improvement over simply taking the first or last frame of the interval.
- **Deferring OCR-based identification.** The proposal's "automatic answer-sheet identification" component turned out to need more groundwork (a labelled dataset of the institute's roll-number/name formats, and a decision on which handwriting-recognition engine to integrate) than the other, more mechanical parts of the pipeline. Rather than block the rest of the prototype on this, the team built the manual assignment path first and designed the schema (`detectedRollNumber`, `detectedName`) so that an OCR step can later populate the same fields the manual path already writes to, with faculty verification staying in the loop exactly as the proposal's fallback-handling plan intended.
- **Keeping the annotated record trustworthy.** Because a complaint doubles as an integrity record of what the student is disputing, the frontend submits a flattened, already-rendered PDF snapshot rather than only the editable annotation JSON, so a later change to the viewer or a client-side bug cannot silently alter what was submitted at complaint time.

6 Limitations

- Answer-sheet–to–student association is manual; there is no automatic OCR/handwriting-recognition step yet, so the "identification accuracy" metric from the proposal cannot be measured at this stage.
- There is no CI/CD pipeline or hosted staging deployment; the system has only been run and tested locally.
- No automated test suite (unit or integration) exists yet; validation so far has been manual, scenario-based testing as described in Section 4.
- The admin role is read-only in the current UI.
- Performance/scalability has not been measured under concurrent load or with large batches of sheets.

7 Conclusion and Future Work

The prototype demonstrates a complete, working loop for the core workflow the proposal set out to build: faculty can turn a recorded video of a physical answer booklet into a digital PDF, attach it to the right student, publish it, and handle any resulting dispute through an auditable annotate-complain-resolve cycle, all without a student needing to visit campus. The main piece of the original proposal not yet delivered is automatic student identification from the scanned sheet; everything around it – the schema fields, the manual verification/override screen, and the downstream publish/notify pipeline – is already in place to receive it.

Planned next steps, in rough priority order, are:

1. Integrate a handwriting-recognition/OCR step (e.g. Tesseract, PaddleOCR, or TrOCR, as originally scoped) to populate `detectedRollNumber`/`detectedName` automatically, with the existing manual-assignment screen repurposed as the faculty verification/correction step.
2. Add automated tests (unit tests for the CV heuristics and page-edit operations, integration tests for the core API routes) and a basic CI pipeline.
3. Move from local processes to a hosted staging deployment, and run the pilot validation plan described in the proposal (10–20 students, timed comparison against the physical process).
4. Benchmark the CV service and backend under concurrent uploads and concurrent PDF downloads to establish real throughput numbers ahead of the proposal's scalability claims.
5. Extend the admin role beyond read-only dashboards (course/exam management, user administration).